https://pubsonline.informs.org/journal/ijoc

INFORMS JOURNAL ON COMPUTING

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

Submitted to *INFORMS Journal on Computing*

ISSN 0899-1499, EISSN 1526-5528

Solving Nonstationary Non-Markovian Queues: A Transfer Learning Neural Network Approach

(Authors' names are not included for peer review)

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and are not intended to be a true representation of the article's final published form. Use of this template to distribute papers in print or online or to submit papers to another non-INFORM publication is prohibited.

Abstract. Empirical studies show that real-world queueing systems, such as contact centers and healthcare facilities, exhibit non-Markovian characteristics, with non-exponential service and abandonment times, and nonstationary dynamics due to time-varying arrivals and staffing. These features make performance analysis and capacity planning highly challenging. Traditional methods such as heavy-traffic approximations and simulation can be computationally intensive or lack sufficient accuracy for practical decision-making. This paper introduces QUANT (*QUeue Analysis via Neural-network Transfer learning*), a neural network framework for analyzing nonstationary non-Markovian queueing systems. QUANT exploits the Markovian counterpart of a complex queue as a structured source model: it first learns fundamental queueing dynamics from large-scale, efficiently generated Markovian data and then uses transfer learning with a much smaller simulated dataset to capture the corrections induced by non-Markovian distributions. This approach substantially reduces training data requirements, enables predictions orders of magnitude faster than simulation, and maintains high accuracy across a range of performance metrics and system configurations. The snapshot-based recurrent structure of QUANT further enables efficient adaptive staffing under multiple service-level constraints. At each decision epoch, QUANT rapidly evaluates candidate staffing levels and selects the minimum feasible capacity required to meet prescribed service targets. By integrating accurate transient prediction with adaptive capacity adjustment, QUANT provides a computationally efficient decision-support framework for dynamic capacity management in complex service operations.

Funding: This research was supported by [grant number, funding agency].

Key words: non-Markovian queues, Nonstationary queues, Staffing in queues, Transfer learning

1. Introduction

The analysis of queueing systems with nonstationary demand and service capacity remains a central challenge in operations research and applied probability. Modern service operations, including cloud computing platforms that provision resources in real time, healthcare systems facing fluctuating

patient arrivals, and call centers with dynamic staffing schedules, are characterized by nonstationary demand and time-varying capacity (Brown et al. 2005, Shi et al. 2014). These systems often exhibit non-Markovian dynamics, as service and abandonment times need not follow exponential distributions, further complicating performance analysis (Brown et al. 2005). At the same time, practical decision-making requires models that can adapt quickly to evolving environments while balancing multiple, sometimes conflicting, performance objectives. Meeting these requirements is particularly challenging in nonstationary non-Markovian queueing systems, where complex dynamics make both performance analysis and decision-making difficult. We next discuss these challenges, the limitations of existing analytical approaches, and the motivation for a learning-based alternative.

1.1. Challenges of Nonstationary Non-Markovian Queues

The analysis of *nonstationary non-Markovian* (NSNM) queueing systems presents formidable challenges that far exceed those encountered in stationary or Markovian settings. The presence of nonexponential service- and abandonment-time distributions breaks the Markov property, precluding the use of convenient state-transition formulations. At the same time, time-varying arrival rates and staffing functions eliminate the notion of equilibrium behavior, forcing the analysis to focus on transient system dynamics that evolve continuously over time. The combination of a non-Markovian probability structure and nonstationary model parameters therefore makes NSNM queues notoriously difficult to analyze, as both stochastic complexity and temporal variability must be captured simultaneously.

Heavy-Traffic Approximations. Heavy-traffic approximations are perhaps the most effective resort currently available for studying NSNM queueing systems. These asymptotic methods, including fluid and diffusion limits, remain among the few analytical tools capable of providing tractable insights when exact analysis is infeasible. Fluid limits describe the system's temporal evolution by averaging out stochastic variability (Liu and Whitt 2012a, Whitt 2006, Liu and Whitt 2011, 2014a). This approach requires solving coupled differential equations that capture both time-varying demand and capacity, often resulting in highly intricate system dynamics. While fluid models provide useful approximations for mean performance, their deterministic nature prevents them from characterizing distributional measures such as the tail probability of the waiting time, $\mathbb{P}(W(t) > \tau)$ (Liu 2018). Their accuracy further deteriorates in moderate- and low-traffic regimes or when the system frequently alternates between underloaded and overloaded states (Liu and Whitt 2012a).

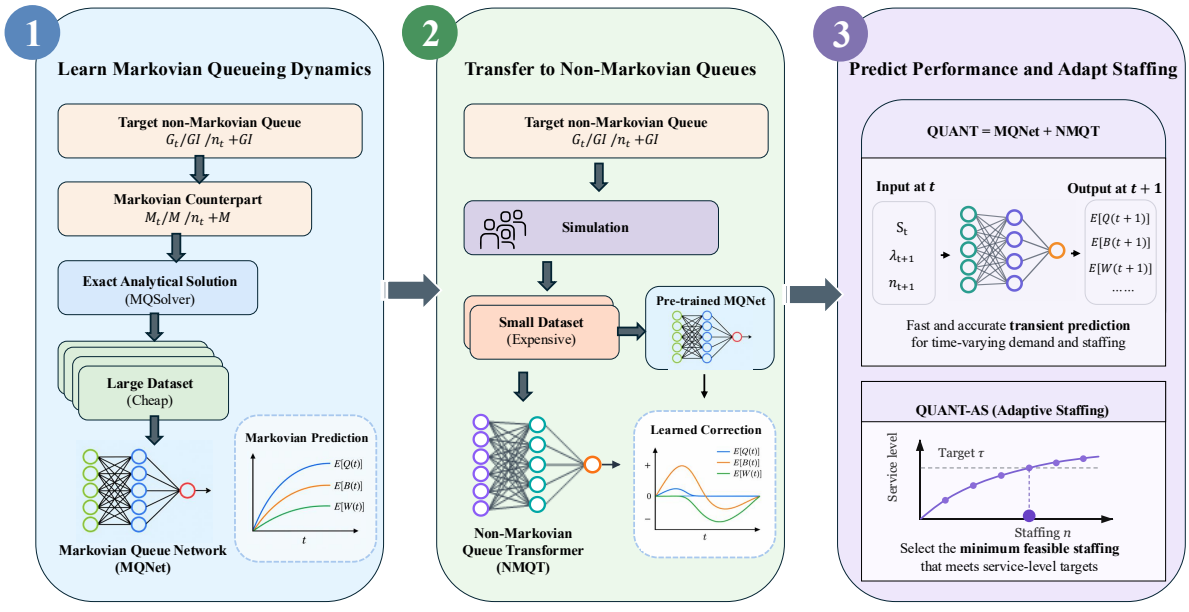
Diffusion limits, derived from the *functional central limit theorem* (FCLT), offer a stochastic refinement to fluid models but remain challenging to derive for general settings. They are most tractable in Markovian systems (Mandelbaum et al. 1998), whereas developing FCLT limits for queues with nonexponential service times and time-varying capacities is exceptionally difficult (Aras et al. 2018, Liu et al. 2016). Even when available, the resulting diffusion models often involve complex stochastic differential equations (Liu and Whitt 2014a) and exhibit limited accuracy for systems of realistic, finite scale.

Importantly, although the target NSNM system is analytically difficult, its Markovian counterpart remains substantially more tractable while preserving the same fundamental demand, capacity, and mean service and abandonment characteristics.

1.2. A Neural-Network Transfer-Learning Approach

To address the need for effective performance analysis and decision making in NSNM queues, we propose a machine learning–based framework that exploits a key structural relationship between a complex non-Markovian queue and its analytically tractable Markovian counterpart. Rather than learning the full NSNM dynamics entirely from expensive simulation, our central idea is to first learn the queueing dynamics that can be captured efficiently from the Markovian counterpart and then use limited simulation data to learn the additional effects induced by non-Markovian service and abandonment distributions.

Figure 1 **Overview of QUANT: Markovian pretraining, non-Markovian adaptation, and adaptive staffing.**



We develop this idea through *Queue Analysis via Neural-network Transfer learning* (QUANT), illustrated in Figure 1. QUANT follows a tractable-baseline-plus-correction learning strategy. For a target NSNM system, we construct a Markovian counterpart in which service and abandonment times are exponential while the time-varying arrival rate, staffing level, and mean service and abandonment times are held identical. Because this counterpart can be analyzed efficiently using the underlying time-varying Markov chain, we can generate a large, high-fidelity training dataset at relatively low computational cost. A base neural network is pretrained on these data to learn the common queueing dynamics associated with time-varying demand and capacity.

Transfer learning is then used to adapt this Markovian baseline to the target non-Markovian system. Specifically, a secondary neural network is trained using a much smaller simulated dataset from the target system, with the Markovian prediction serving as a structured baseline from which to learn the distribution-induced correction. In this way, expensive simulation is concentrated on learning the effects that distinguish the target system from its tractable counterpart, rather than relearning the entire queueing dynamics from scratch. Once trained, QUANT recurrently predicts transient service-level metrics, including queue length, waiting time, and delay probabilities, under time-varying demand and staffing. Its low inference cost further enables rapid evaluation of alternative capacity levels, supporting adaptive staffing under prescribed service-level constraints.

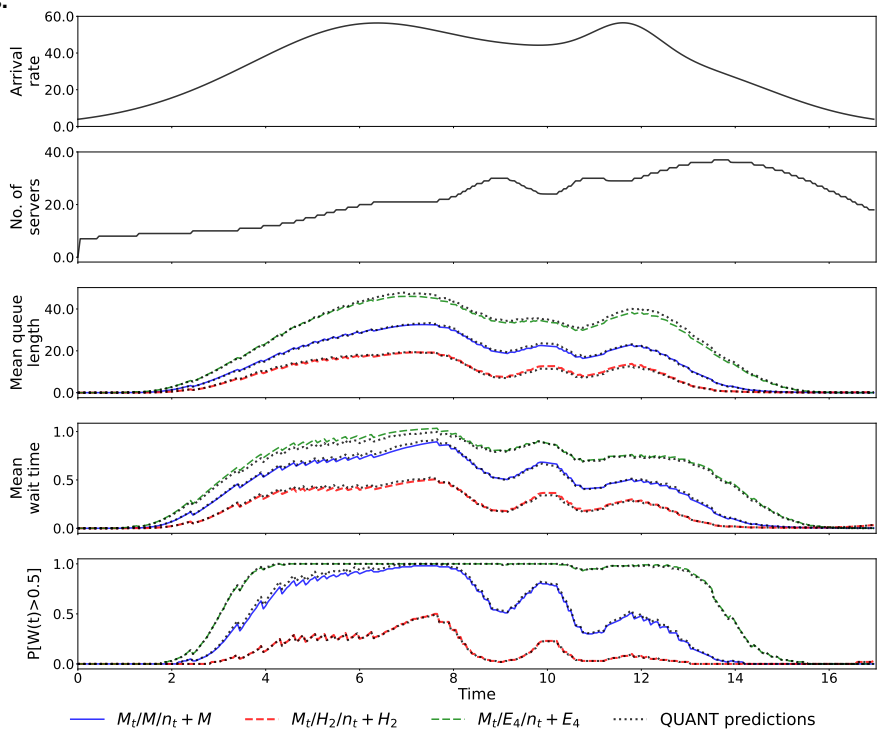
This design reflects the structure of many practical service systems. For a given customer class, daily arrival-rate and staffing functions can vary substantially from day to day, whereas the underlying service- and abandonment-time distributions are typically more stable. Accordingly, for a specified pair of service- and abandonment-time distributions, a trained QUANT model can be reused across varying arrival-rate and staffing functions, allowing the model to support a broad range of operating conditions without retraining for each demand or staffing profile.

1.3. A Motivating Example

We illustrate QUANT's performance on several nonstationary queueing systems. We consider a nonstationary Markovian queueing system $M_t/M/n_t + M$ (with M denoting exponential distributions and subscript t denoting time-varying arrival and staffing) and two non-Markovian variants: $M_t/H_2/n_t + H_2$, with hyperexponential service and abandonment times, and $M_t/E_4/n_t + E_4$, with Erlang service and abandonment times. All three systems are special cases of the general $G_t/GI/n_t + GI$ type; detailed definitions are introduced in Section 3. The systems have identical mean service and abandonment times ($1/\mu = 1$ and $1/\theta = 1$), but differ substantially in distributional variability, with squared coefficients of variation $\mathrm{SCV}_{H_2} = 4$, $\mathrm{SCV}_M = 1$, and $\mathrm{SCV}_{E_4} = 1/4$. Complete distributional specifications are provided in Appendix E.

QUANT first learns the common queueing dynamics from the corresponding $M_t/M/n_t + M$ Markovian system and then uses transfer learning to capture the distribution-induced corrections for the two non-Markovian systems. As illustrated in Figure 2, QUANT closely tracks the simulation ground truth across multiple performance metrics, including mean queue length $\mathbb{E}[Q(t)]$, mean waiting time $\mathbb{E}[W(t)]$, and the tail probability of delay (TPoD) $\mathbb{P}(W(t) > 0.5)$.

Figure 2 **QUANT predictions (dotted lines) versus simulation ground truth for Markovian and non-Markovian systems.**



QUANT accurately captures both the magnitude and timing of performance changes as the systems transition between underloaded and overloaded regimes. Particularly noteworthy is its ability to predict tail-delay probabilities, which depend on complex interactions among arrival variability, service-time distributions, and abandonment behavior. Section 4 provides extensive numerical experiments across different system scales, demand patterns, and distributional settings, while Section 5 demonstrates how QUANT's fast inference can support adaptive staffing under multiple service-level constraints.

1.4. Contributions and Organization

We summarize the main contributions of this paper.

- **Deep learning for nonstationary non-Markovian queues.** We develop QUANT, a deep learning framework for predicting transient performance in NSNM queues with time-varying

arrivals and staffing and general service and abandonment times. QUANT recurrently predicts key service-level metrics, including mean waiting time, mean queue length, abandonment probability, and tail delay probabilities.

- **New architecture for time-varying staffing.** Unlike previous methods that rely on the point-wise steady-state (PSS) approximation under constant staffing (Garyfallos et al. 2025), QUANT does not depend on PSS. This enables time-varying staffing and a broader class of systems, including $G_t/GI/n_t$ queues without customer abandonment, while reducing costly steady-state simulation data.
- **Transfer learning through a tractable Markovian baseline.** QUANT learns common queueing dynamics from an efficiently solvable Markovian counterpart and uses a much smaller target-system simulation dataset to learn the effects induced by the target non-Markovian distributions. This strategy reduces expensive target-system training data by up to 99% in our experiments.
- **Adaptive staffing under service-level constraints.** We develop QUANT-based Adaptive Staffing (QUANT-AS), which rapidly evaluates candidate staffing levels and selects the minimum feasible capacity satisfying prescribed service-level targets at each decision epoch. This translates QUANT's fast transient prediction into real-time capacity decision support.

Organization of the paper. In Section 2, we review the relevant literature. In Section 3, we introduce QUANT, including its architecture, data-generation strategy, and transfer-learning procedure. In Section 4, we evaluate its prediction accuracy, computational efficiency, and data efficiency. In Section 5, we develop QUANT-AS for adaptive staffing under multiple service-level constraints. Finally, Section 6 concludes the paper and discusses future research directions.

2. Related Literature

The present work builds upon and extends research in three primary domains: the performance analysis of NSNM queueing systems, time-varying staffing to stabilize performance in nonstationary queues, and the application of machine learning techniques to queueing theory. This section reviews the relevant literature in these areas.

2.1. Nonstationary Non-Markovian Queue: Performance Analysis and Capacity Planning

The analysis of queueing systems with nonstationary parameters, such as nonstationary arrival rates and time-varying staffing levels, has been an important area of queueing theory due to its practical relevance in real-world service systems like call centers and healthcare facilities. Common approaches for analyzing nonstationary queues often involve fluid and diffusion approximations.

Mandelbaum et al. (1998) investigated many-server heavy-traffic limits for Markovian queueing networks with nonstationary parameters. Liu and Whitt (2012a, 2014a) developed fluid and FCLT limits for $G_t/GI/s_t+GI$ queues with nonexponential distributions and nonstationary arrivals. See Whitt (2018) for a broad overview of research on time-varying queues.

Our work is related to the literature on the staffing problems of queueing systems with time-varying arrivals. Pointwise stationary approximation (Green et al. 2007) has been proven useful in staffing systems with shorter service times, following the basic idea of approximating a nonstationary model at each time point by a stationary model. The modified-offered-load (MOL) approximation has been developed to design staffing functions to control performance functions, including the probability of delay (PoD), mean waiting time, and probability of abandonment (PoA) (Jennings et al. 1996, Liu and Whitt 2012b, 2014b, 2017). Also see He et al. (2016), Sun and Liu (2021) for the extension to queues with non-Poisson arrival processes. Feldman et al. (2008) developed a simulation-based iterative staffing algorithm (ISA) to control the PoD. The idea of ISA was later extended by Defraeye and Van Nieuwenhuyse (2013), Konrad and Liu (2023) to treat the TPoD. Unlike ISA, Liu (2018) was the first to develop an analytic staffing method for nonstationary queues with nonexponential abandonment times to stabilize the TPoD and prove the asymptotic correctness of the staffing algorithm. Liu et al. (2022) further extended results in Liu (2018) to queues having multiple customer classes, each having its own TPoD target. Distinct from the above literature, our present study conducts performance analysis and develops staffing functions for the more general NSNM queues. In addition, we aim to satisfy constraints involving multiple performance metrics.

2.2. Machine Learning in Queueing Systems

Recent developments in deep learning have fundamentally transformed the analysis and optimization of queueing systems, moving beyond traditional analytical approaches to embrace data-driven methodologies. Theoretical advances have established deep learning's capability to approximate complex queueing behaviors (Baron et al. 2022, 2024, Sherzer et al. 2025). Garyfallos et al. (2025) developed NeuraliNQ, a neural-network-based prediction method for non-Markovian queues under a constant staffing level. However, when applied to general NSNM queueing systems, NeuraliNQ faces three key limitations. First, its reliance on PSS leads to substantial simulation overhead. Second, this reliance restricts the method to systems with customer abandonment; without abandonment, steady-state distributions do not exist under overloaded conditions. Finally, NeuraliNQ cannot accommodate time-varying staffing functions, which prevents it from being used for staffing optimization.

QUANT extends NeuraliNQ through transfer learning from a tractable Markovian counterpart. It first pretrains MQNet on efficiently generated Markovian data, then trains NMQT on a small target-system simulation dataset to adapt the pretrained predictions to non-Markovian dynamics. By removing reliance on PSS, QUANT avoids costly steady-state simulations and accommodates time-varying staffing functions $n(t)$ and systems without customer abandonment. Its fast transient predictions further support adaptive staffing under multiple service-level constraints.

3. QUANT Methodology

The QUANT framework is designed for the real-time performance analysis and adaptive decision support of general NSNM queueing systems. In particular, we consider the $G_t/GI/n_t+GI$ system which is characterized by a general arrival process with a nonstationary rate $\lambda(t)$ (the G_t), time-varying number of servers $n(t)$ (the n_t), *independent and identically distributed* (i.i.d.) service times following a general distribution G (the GI) with mean $1/\mu$ for $\mu > 0$, and customer abandonment according to i.i.d. random variables following a general distribution F (the $+GI$) with mean $1/\theta$ for $\theta \geq 0$. When $\theta = 0$, our model reduces to the $G_t/GI/n_t$ queue without customer abandonment. All customers are served under the *first-come, first-served* (FCFS) discipline. For a given pair of service- and abandonment-time distributions, QUANT learns a reusable transient prediction model across varying arrival-rate and staffing functions $\lambda(t)$ and $n(t)$. Our objective is to predict the transient trajectory of key *service level* (SL) metrics $\eta(t)$, such as the mean waiting time, mean queue length, and TPoD over a finite horizon $t \in [0,T]$. Let $Q(t)$ and $B(t)$ denote the numbers of customers in the waiting queue and in service, and let $W(t)$ be the waiting time at time t.

3.1. Structural Design and Architecture of the QUANT Framework

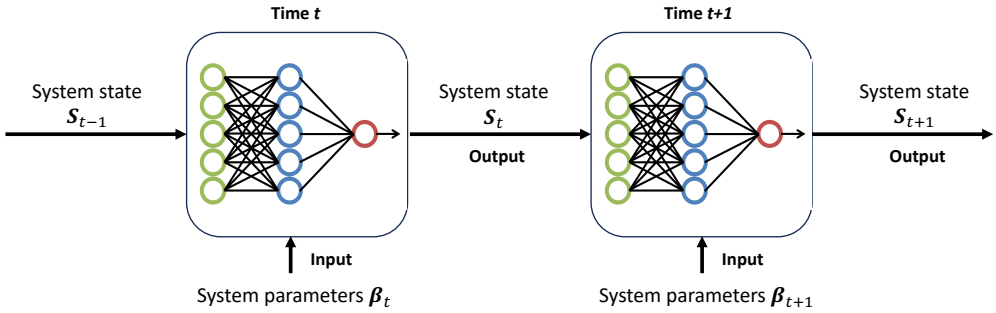
In the rest of this section, we describe the structural design of QUANT at three progressively finer levels of detail: *macroscopic*, *mesoscopic*, and *microscopic*.

3.1.1. Macroscopic level: At the *macroscopic* level, QUANT is a recurrent neural network that predicts the system state at the next time step based on the current time step. For each time t, QUANT serves as a function $\mathcal{F}$ that computes the predictive state $\mathbf{S}_{t+1}$ using (i) the present predictive state $\mathbf{S}_t$, and (ii) the next-step model input $\boldsymbol{\beta}_{t+1}$ (e.g., $\lambda(t+1)$ and $n(t+1)$). Symbolically, the heart of QUANT is in the form of

$$\mathbf{S}_{t+1} = \mathcal{F}(\mathbf{S}_t, \boldsymbol{\beta}_{t+1}), \quad t = 0, 1, 2, \ldots, \tag{1}$$

where $\mathbf{S}_t$ is the predictive state representation at time t, and $\boldsymbol{\beta}_{t+1}$ encompasses the external model inputs for the interval $(t, t+1]$, such as the arrival rate $\lambda(t+1)$ and number of servers $n(t+1)$.

Figure 3 QUANT at the macroscopic level: A feedforward topology of time-indexed RNN.



Predictive State Representation: For a general NSNM queue, an exact stochastic state may require substantially richer information, such as the elapsed service and waiting times of individual customers (Whitt 2006). We therefore do not assert that a low-dimensional snapshot is a Markov-sufficient state. Instead, QUANT uses a parsimonious predictive state representation consisting of aggregate performance measures that summarize congestion in both the waiting and service components of the system. Specifically, we use

$$\mathbf{S}_t \equiv (\mathbb{E}[Q(t)], \mathbb{E}[B(t)], \mathbb{E}[W(t)]), \tag{2}$$

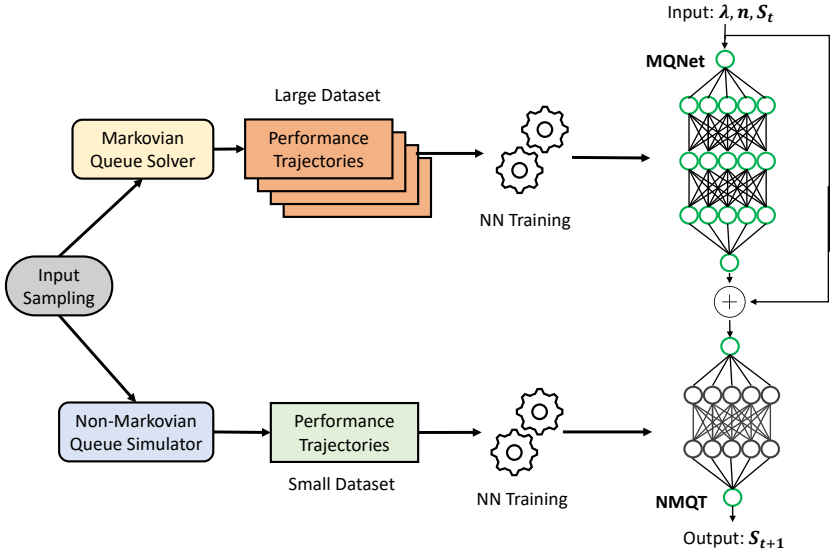
which tracks the means of the queue length, number of busy servers, and waiting time at time t.

This multi-faceted representation captures information from both the waiting area (queue length and waiting time) and the service area (server occupancy $\mathbb{E}[B(t)]/n(t)$). This is particularly useful for systems with varying scales or those transitioning between underloaded and overloaded states, where different metrics may be informative about future system dynamics. Together with the anticipated arrival rate $\lambda(t+1)$ and planned server capacity $n(t+1)$ contained in $\boldsymbol{\beta}_{t+1}$, these quantities provide a compact input for predicting next-period performance.

This representation deliberately trades exact state sufficiency for computational parsimony; its adequacy is therefore a predictive approximation property rather than a Markovian assumption. As demonstrated in Section 4, this compact representation provides accurate transient predictions across the systems considered in our experiments. The predictive state representation can also be expanded to incorporate additional performance metrics, such as TPoD, as demonstrated in Section 1.3.

3.1.2. Mesoscopic level: Zooming into the one-step prediction module, we now explain QUANT's *mesoscopic*-level architecture. As illustrated in Figure 4, our one-step prediction model comprises two submodules: a *Markovian queue network* (MQNet) and a *non-Markovian queue transformer* (NMQT). Our key idea leverages transfer learning, a fundamental paradigm in deep

Figure 4 QUANT at the mesoscopic level: Transfer learning from Markovian queue to non-Markovian queue.



neural networks that enables knowledge learned from a source domain to improve learning in a target domain with limited data (Pratt et al. 1991, Caruana 1997, Pan and Yang 2010). We first pretrain MQNet to capture the queueing dynamics of an $M_t/M/n_t+M$ system across varying arrival-rate and staffing functions. This system has the same arrival rate $\lambda(t)$, staffing level $n(t)$, mean service time $1/\mu$, and mean abandonment time $1/\theta$ as the target NSNM $G_t/GI/n_t+GI$ system, but replaces the target service- and abandonment-time distributions with exponential distributions. We refer to this system as the *Markovian counterpart*. Because its transient performance can be computed efficiently, MQNet can learn common queueing dynamics from a large, high-fidelity dataset without relying on expensive simulation of the target non-Markovian system.

NMQT then adapts this Markovian baseline to the specified target service- and abandonment-time distributions. As illustrated in Figure 4, MQNet takes λ_{t+1}, n_{t+1}, and $\mathbf{S}_t$ as inputs and produces a Markovian prediction of $\mathbf{S}_{t+1}$. NMQT takes the same inputs together with the MQNet output, using the Markovian prediction as a structured baseline for learning the additional effects induced by the target non-Markovian distributions. Thus, rather than learning the entire target transition from scratch, NMQT focuses on the deviation from the Markovian benchmark. This provides a natural inductive bias that can improve data efficiency (Perdikaris et al. 2017, Meng and Karniadakis 2020).

We consider several transfer-learning strategies, including fine-tuning the entire MQNet and fine-tuning only its final layers. In the implementation used throughout the paper, we freeze the pretrained MQNet and train the additional NMQT module using a much smaller simulation dataset from the target non-Markovian system. This separation preserves the common queueing dynamics learned from the Markovian counterpart while concentrating expensive simulation data on adapting the model to the target distributional setting.

3.1.3. Microscopic level: At the microscopic level, MQNet employs a tandem structure with two specialized networks, $\mathcal{F}^Q$ and $\mathcal{F}^W$, that process count-based and time-based metrics separately. We denote the one-step-ahead Markovian baseline prediction from MQNet by $\widetilde{\mathbf{S}}_{t+1}$, whose count-based and waiting-time components are given by

$$\widetilde{\mathbf{S}}^Q_{t+1} = \mathcal{F}^Q(\mathbf{S}_t, \boldsymbol{\beta}_{t+1}), \tag{3}$$

$$\widetilde{\mathbf{S}}^W_{t+1} = \mathcal{F}^W(\widetilde{\mathbf{S}}^Q_{t+1}, \mathbf{S}^W_t, \boldsymbol{\beta}_{t+1}), \tag{4}$$

where $\mathbf{S}_t = (\mathbf{S}^Q_t, \mathbf{S}^W_t)$, $\mathbf{S}^Q_t = (\mathbb{E}[Q(t)], \mathbb{E}[B(t)])$, and $\mathbf{S}^W_t = (\mathbb{E}[W(t)])$.

For transfer learning, NMQT introduces two corresponding networks, $\mathcal{G}^Q$ and $\mathcal{G}^W$, which additionally incorporate the Markovian baseline predictions produced by MQNet:

$$\mathbf{S}^Q_{t+1} = \mathcal{G}^Q(\mathbf{S}_t, \boldsymbol{\beta}_{t+1}, \widetilde{\mathbf{S}}^Q_{t+1}), \tag{5}$$

$$\mathbf{S}^W_{t+1} = \mathcal{G}^W(\mathbf{S}^Q_{t+1}, \mathbf{S}^W_t, \boldsymbol{\beta}_{t+1}, \widetilde{\mathbf{S}}^W_{t+1}). \tag{6}$$

Thus, NMQT predicts the target non-Markovian dynamics conditional on both the current predictive state and the corresponding Markovian prediction. This architecture allows the pretrained MQNet to provide a structured baseline, while NMQT adapts that baseline to the specified non-Markovian service- and abandonment-time distributions.

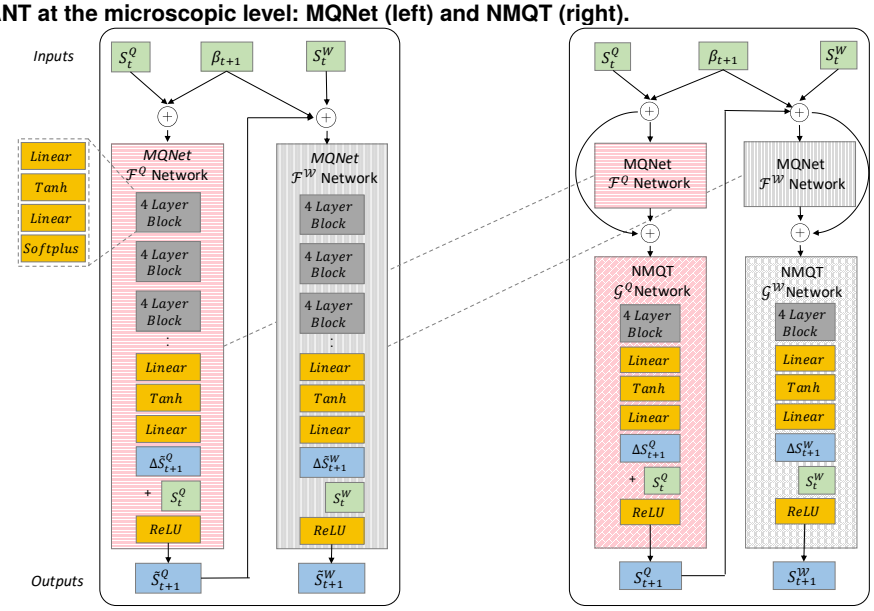
We next provide the implementation details of the neural networks at the microscopic level. Figure 5 illustrates the internal architectures of MQNet and NMQT. To describe their implementations, we use f^Q, f^W and g^Q, g^W to denote the corresponding feedforward cores of MQNet and NMQT, respectively. These cores share the same basic architecture, consisting of repeated blocks of linear transformations with alternating Tanh and Softplus activations. Rather than directly producing the next predictive state, they predict the corresponding one-step state increments:

$$\begin{aligned} \Delta\widetilde{\mathbf{S}}^Q_{t+1} &= f^Q(\mathbf{S}_t, \boldsymbol{\beta}_{t+1}), & \Delta\mathbf{S}^Q_{t+1} &= g^Q(\mathbf{S}_t, \boldsymbol{\beta}_{t+1}, \widetilde{\mathbf{S}}^Q_{t+1}), \\ \Delta\widetilde{\mathbf{S}}^W_{t+1} &= f^W(\widetilde{\mathbf{S}}^Q_{t+1}, \mathbf{S}^W_t, \boldsymbol{\beta}_{t+1}), & \Delta\mathbf{S}^W_{t+1} &= g^W(\mathbf{S}^Q_{t+1}, \mathbf{S}^W_t, \boldsymbol{\beta}_{t+1}, \widetilde{\mathbf{S}}^W_{t+1}). \end{aligned} \tag{7}$$

The complete state-transition mappings then combine these increments with the current state through residual updates:

$$\widetilde{\mathbf{S}}_{t+1} = \mathrm{ReLU}\left(\mathbf{S}_t + \Delta\widetilde{\mathbf{S}}_{t+1}\right), \qquad \mathbf{S}_{t+1} = \mathrm{ReLU}(\mathbf{S}_t + \Delta\mathbf{S}_{t+1}). \tag{8}$$

Figure 5 **QUANT at the microscopic level: MQNet (left) and NMQT (right).**



The residual updates retain information from the current system state, while the final ReLU activation enforces nonnegativity of the predicted queueing states. The alternating Tanh and Softplus activations used within the feedforward cores provide smooth nonlinear transformations, supporting stable training of deeper network structures over recurrent state transitions.

3.2. Data Generation and Model Training

We next describe the data-generation and training procedure for QUANT. The procedure follows the structural decomposition introduced in Section 3.1.2: we first use the tractable Markovian counterpart to generate a large, high-fidelity dataset for learning common queueing dynamics across varying arrival-rate and staffing functions, and then use a much smaller simulated dataset from the target non-Markovian system to adapt the Markovian baseline to the specified service- and abandonment-time distributions.

3.2.1. Step 1: Generating Markovian queueing data. We first generate training data for the Markovian counterpart $M_t/M/n_t+M$. For each sampled pair of arrival-rate and staffing trajectories, $\lambda(t)$ and $n(t)$, we compute the corresponding transient queueing-performance trajectories using MQSolver. Because the Markovian counterpart can be analyzed efficiently through its underlying time-varying Markov-chain structure, these training targets can be generated substantially more efficiently than by Monte Carlo simulation.

MQSolver computes the transient distribution of the number of customers in the system, $\pi_x(t) \equiv \mathbb{P}(X(t)=x)$, using the time-varying birth-and-death structure of the Markovian counterpart. Con-

ditional waiting-time distributions are then obtained using the phase-type representation described in Appendix B. The unconditional waiting-time distribution follows from

$$\mathbb{P}(W(t) \le \tau) = \sum_{x} \mathbb{P}(W(t) \le \tau \mid X(t) = x)\pi_x(t), \qquad \tau > 0,\ t \in [0,T]. \tag{9}$$

This allows MQSolver to generate both mean and distributional service-level metrics, including waiting-time and TPoD measures. Complete algorithmic details are provided in Appendix B.

3.2.2. Step 2: Sampling nonstationary input trajectories.

Generating Input Trajectories $\lambda(t)$ and $n(t)$: To pretrain MQNet, we require realistic and diverse nonstationary trajectories for the arrival rate $\lambda(t)$ and service capacity $n(t)$. We generate both trajectories independently using additive Gaussian basis functions (Bilancia et al. 2021). Specifically, we define

$$P(t) = \sum_{k=1}^{K} c_k \psi_k(t), \qquad \psi_k(t) = \frac{1}{\sigma_k \sqrt{2\pi}} \exp\left(-\frac{(t-\mu_k)^2}{2\sigma_k^2}\right), \tag{10}$$

where K is the total number of Gaussian components, $k = 1, \ldots, K$ indexes the individual components, and c_k, μ_k, and σ_k control the intensity, location, and width of component k, respectively. The resulting profile is rescaled as

$$b(t) = \frac{M}{\max_{s\in[0,T]} P(s)} P(t), \tag{11}$$

where M specifies the maximum level of the generated trajectory. Independent parameter draws are used to construct $\lambda(t)$ and $n(t)$, with the latter discretized to obtain an integer-valued service capacity. Complete sampling specifications are provided in Appendix C.1.

3.2.3. Step 3: Sampling small-scale non-Markovian queueing data via simulation. We next generate a much smaller dataset from the target NSNM system. For a specified pair of service- and abandonment-time distributions G and F, we simulate the $G_t/GI/n_t + GI$ system under a subset of the arrival-rate and staffing scenarios generated in Step 2. These simulations provide high-fidelity observations of the target dynamics that are not captured by the Markovian counterpart, particularly the effects associated with the shapes of the service- and abandonment-time distributions beyond their means.

Because Monte Carlo simulation of the target NSNM system is substantially more expensive than solving its Markovian counterpart, only a small number of these trajectories are generated. The objective is therefore not to relearn the complete queueing dynamics from simulation, but to provide sufficient target-system information for adapting the pretrained Markovian baseline. Complete simulation specifications are provided in Appendix C.2.

3.2.4. Step 4: Transfer learning for training the non-Markovian transformer. We finally transfer the information learned by MQNet to the target non-Markovian system. The procedure consists of two stages:

- **Markovian pretraining:** The complete MQNet architecture ($\mathcal{F}^Q$ and $\mathcal{F}^W$) is trained on the large dataset generated by MQSolver for the $M_t/M/n_t + M$ Markovian counterpart. This stage learns common transient queueing dynamics across varying arrival-rate and staffing functions.
- **Non-Markovian adaptation:** The pretrained MQNet is frozen, and NMQT is trained using the much smaller simulation dataset from the target $G_t/GI/n_t + GI$ system. NMQT receives the Markovian prediction as a structured baseline and learns to adapt it to the specified target service- and abandonment-time distributions.

As described in Section 3.1.2, we also consider alternative transfer strategies, including fine-tuning the entire MQNet or only its final layers. In the implementation used throughout the paper, however, we freeze MQNet and train the additional NMQT module. For training with limited and potentially noisy simulation data, NMQT is trained with a lower learning rate than MQNet. Complete hyperparameter settings are provided in Appendix D.

4. Numerical Experiments

We conduct extensive numerical experiments to evaluate QUANT along three dimensions: prediction accuracy across different queueing environments, computational efficiency, and the amount of target non-Markovian simulation data required for effective transfer. In particular, the experiments examine whether the common queueing dynamics learned from the Markovian counterpart can be effectively transferred to systems with different service- and abandonment-time distributions using only limited target-system data.

We use Monte Carlo simulation as the ground truth. Prediction accuracy is measured using the *time-averaged mean squared error* (TAMSE):

$$\mathcal{E}^\eta \equiv \frac{1}{J+1}\sum_{j=0}^{J}\left(e_j^\eta\right)^2, \qquad e_j^\eta \equiv \hat{\eta}(t_j) - \eta(t_j), \tag{12}$$

where $\hat{\eta}(t_j)$ and $\eta(t_j)$ denote the predicted and simulated values of service-level metric η at time point t_j, respectively, with $t_j = j\Delta t$, $j = 0, \ldots, J$, and $J = T/\Delta t$.

4.1. Experimental Setup

We follow the input-generation and simulation procedures described in Section 3, with complete implementation details provided in Appendix C. For the service and abandonment times, we consider three queueing configurations: exponential ($M_t/M/n_t+M$), hyperexponential ($M_t/H_2/n_t+H_2$), and Erlang ($M_t/E_4/n_t+E_4$). These configurations allow us to evaluate QUANT across systems with substantially different distributional variability while holding the corresponding means fixed.

MQNet is pretrained on 10,000 Markovian trajectories, with 20% held out for testing, and contains approximately 174 million parameters across the $\mathcal{F}^Q$ and $\mathcal{F}^W$ networks. For each target non-Markovian setting, NMQT is trained using only 100 simulated trajectories, corresponding to 1% of the MQNet training-data size. All models are trained using the L1 loss and the Adam optimizer. Complete hyperparameter settings and training procedures are provided in Appendix D.

4.2. Prediction Accuracy and Robustness

Table 1 reports QUANT's prediction accuracy across different system scales and service- and abandonment-time distributions. We evaluate four performance metrics: mean queue length $\mathbb{E}[Q(t)]$, mean number of busy servers $\mathbb{E}[B(t)]$, mean waiting time $\mathbb{E}[W(t)]$, and the tail probability of delay $\mathbb{P}(W(t) > 1)$.

Scale comparisons are conducted under the Markovian $M_t/M/n_t + M$ setting. System scale is characterized by the maximum staffing level $n_{\max} \equiv \max_{t\in[0,T]} n(t)$. We classify systems as small ($n_{\max} \le 41$), medium ($41 < n_{\max} \le 72$), and large ($72 < n_{\max} \le 100$). Across system scales, QUANT maintains consistently low prediction errors, with TAMSE ranging from 0.038 to 0.118 across the four metrics. Prediction accuracy also remains stable as system scale increases, indicating that the learned representation is not tied to a narrow operating scale.

Table 1 TAMSE accuracy across system configurations.

Category	Configuration	$\mathbb{E}[Q(t)]$	$\mathbb{E}[B(t)]$	$\mathbb{E}[W(t)]$	$\mathbb{P}(W(t) > 1)$
System Scale	Small	0.118	0.103	0.078	0.046
	Medium	0.103	0.089	0.068	0.043
	Large	0.087	0.074	0.057	0.038
Distribution	$M_t/M/n_t + M$	0.095	0.083	0.062	0.041
	$M_t/H_2/n_t + H_2$	0.098	0.085	0.064	0.043
	$M_t/E_4/n_t + E_4$	0.097	0.084	0.063	0.042

Scale comparisons are conducted under the Markovian setting, and distributional comparisons at the baseline system scale. TAMSE is reported separately for each metric.

More importantly for the transfer-learning architecture, prediction errors remain similar across the exponential, hyperexponential, and Erlang settings. For example, TAMSE for $\mathbb{E}[Q(t)]$ ranges only from 0.095 to 0.098, while TAMSE for $\mathbb{P}(W(t) > 1)$ ranges from 0.041 to 0.043. Thus, despite substantial differences in distributional variability, the Markovian baseline can be effectively adapted to the target non-Markovian systems using limited target-system data.

Figure 2 further illustrates the full prediction trajectories under the exponential, hyperexponential, and Erlang service- and abandonment-time distributions. As discussed in Section 1.3, QUANT captures both the magnitude and timing of changes in transient system performance, including tail-delay probabilities.

4.3. Runtime Efficiency

We next compare the runtime of QUANT, MQSolver, and Monte Carlo simulation for trajectories of duration $T = 17$, corresponding to 340 time steps. All computations are performed on a single Intel Xeon E5-2686 v4 CPU core at 2.30 GHz and are reported in CPU time. QUANT completes a full trajectory evaluation in approximately 0.35 seconds, compared with approximately 60 seconds for MQSolver and 900 seconds for Monte Carlo simulation.

The computational advantage is particularly important when the performance model must be evaluated repeatedly, as in the adaptive staffing application developed in Section 5. Once trained, QUANT replaces repeated numerical solution or simulation with inexpensive neural-network inference. Generating the training data requires approximately 7 CPU-days for the Markovian systems and approximately 1 CPU-day for the non-Markovian transfer data. For completeness, training MQNet requires approximately 30 minutes on a Tesla V100 SXM2 16GB GPU, while training NMQT requires approximately 10 minutes.

4.4. Transfer-Learning Data Efficiency

A central motivation for QUANT is to reduce the amount of expensive target-system simulation required to construct an accurate predictor. We therefore evaluate NMQT across different target-system training-data sizes for the $M_t/H_2/n_t + H_2$ system. Table 2 reports TAMSE separately for $\mathbb{E}[Q(t)]$, $\mathbb{E}[B(t)]$, $\mathbb{E}[W(t)]$, and $\mathbb{P}(W(t) > 1)$.

QUANT maintains strong prediction accuracy even when the target-system training data are substantially reduced. With only 100 non-Markovian trajectories—1% of the full dataset—TAMSE increases by at most 0.005 for each of the four metrics relative to using all 10,000 trajectories. Performance remains reasonably stable with 50 trajectories (i.e., 0.5% of full dataset), whereas a

Table 2 Prediction accuracy (TAMSE) across different amounts of non-Markovian training data used for transfer learning. All values represent time-averaged mean squared errors relative to simulation ground truth.

Dataset	100%	50%	25%	10%	5%	1%	0.5%	0.25%
Trajectories	10000	5000	2500	1000	500	100	50	25
$\mathbb{E}[Q(t)]$	0.096	0.097	0.097	0.098	0.098	0.101	0.121	0.200
$\mathbb{E}[B(t)]$	0.083	0.084	0.084	0.085	0.086	0.088	0.105	0.173
$\mathbb{E}[W(t)]$	0.063	0.063	0.064	0.064	0.065	0.066	0.079	0.130
$\mathbb{P}(W(t) > 1)$	0.041	0.042	0.042	0.043	0.043	0.044	0.052	0.086

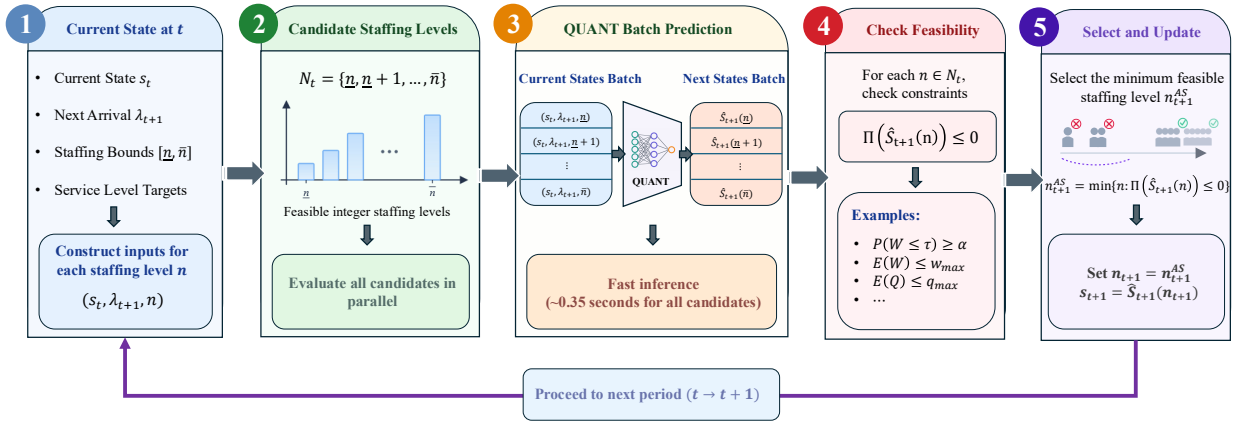
more pronounced deterioration occurs when the dataset is reduced to 25 trajectories (i.e., 0.25% of full dataset).

These results provide direct empirical support for the central transfer-learning mechanism of QUANT. Much of the transient queueing structure can be learned from the tractable Markovian counterpart, leaving the smaller target-system dataset to adapt this baseline to the distribution-induced effects of the non-Markovian system. Generating 100 target-system trajectories requires approximately 1 CPU-day, compared with approximately 100 CPU-days for the full 10,000-trajectory dataset. Thus, in this experiment, QUANT reduces the expensive target-system simulation requirement by 99% while incurring only a small loss in prediction accuracy.

5. QUANT-based Adaptive Staffing under Service-Level Constraints

We develop *QUANT-based Adaptive Staffing* (QUANT-AS) for sequential staffing adjustments under multiple service-level (SL) constraints. QUANT-AS uses predicted queueing performance to guide each adjustment; it does not jointly optimize staffing costs over the full horizon. Throughout this section, t indexes discrete decision epochs, and T denotes the number of epochs.

Figure 6 Adaptive staffing under service-level constraints.



5.1. The QUANT-AS Algorithm

Starting from a baseline staffing trajectory $\{n_t\}_{t=1}^{T}$, QUANT-AS retains the baseline staffing at each epoch if it is predicted to satisfy all prescribed SL constraints. Otherwise, it evaluates candidate staffing levels through batch inference and selects the smallest predicted-feasible level, as illustrated in Figure 6. If no candidate is predicted to be feasible, it uses the upper staffing bound as a fallback, without guaranteeing constraint satisfaction. The predicted state under the selected staffing is then propagated to the next epoch.

One-Step Predictor and State Representation: Let $\mathbf{s}_t \in \mathbb{R}^d$ denote the predictive system state at time t, summarizing the queueing dynamics. At each decision epoch, the QUANT input is constructed as $\tilde{\mathbf{S}}_t^a = [\,\mathbf{s}_t^\top, \lambda_{t+1}, n_{t+1}\,] \in \mathbb{R}^{1\times(d+2)}$, where λ_{t+1} and n_{t+1} denote the anticipated arrival rate and staffing level for the next period, respectively.

Let b denote the batch size, i.e., the number of candidate inputs evaluated simultaneously. QUANT defines a vectorized mapping $\tilde{\mathbf{S}}_{t+1}^o = \mathcal{F}(\tilde{\mathbf{S}}_t^a) : \mathbb{R}^{b\times(d+2)} \to \mathbb{R}^{b\times d}$, which maps each augmented input state to a d-dimensional next-period predictive state. The vectorized implementation allows QUANT to evaluate multiple candidate staffing levels simultaneously in a single forward pass.

Constraint Policy Function: Suppose there are k SL metrics $\eta_1(t), \ldots, \eta_k(t)$ with corresponding upper bounds $\bar{\eta}_1, \ldots, \bar{\eta}_k$. We require $\eta_i(t) \le \bar{\eta}_i,\ i = 1, \ldots, k,\ t \in [0, T]$. Typical constraints include bounds on mean waiting time $\mathbb{E}[W(t)]$, mean queue length $\mathbb{E}[Q(t)]$, and tail probability of delay $\mathbb{P}(W(t) > w)$. Let $\Pi(\tilde{\mathbf{S}}_{t+1}^o)$ denote a constraint-evaluation function that maps a predicted next-period system state to a scalar measure of constraint violation. Specifically, we define

$$\Pi(\tilde{\mathbf{S}}_{t+1}^o) = h(\eta_1(t+1) - \bar{\eta}_1, \ldots, \eta_k(t+1) - \bar{\eta}_k)\,,$$

where $h(\cdot)$ is monotone increasing in each argument. Although this formulation permits a general aggregation function, throughout the remainder of the paper we use $h(x_1, \ldots, x_k) = \max\{x_1, \ldots, x_k\}$, so that $\Pi(\tilde{\mathbf{S}}_{t+1}^o) \le 0$ if and only if all prescribed SL constraints are satisfied.

Batch Evaluation of Candidate Staffing Levels. At each decision epoch, QUANT-AS first evaluates the baseline staffing n_{t+1}. If it satisfies the prescribed SL constraints, QUANT-AS retains the baseline level. Otherwise, it constructs the candidate set $\mathcal{N}_t = \{\underline{n}, \underline{n}+1, \ldots, \bar{n}\}$. For batch evaluation, the current state $\mathbf{s}_t$ and anticipated arrival rate λ_{t+1} are replicated across all candidate staffing levels:

$$\mathbf{S}_t = \mathbf{1}_{|\mathcal{N}_t|}\mathbf{s}_t^\top \in \mathbb{R}^{|\mathcal{N}_t|\times d}, \qquad \boldsymbol{\lambda}_{t+1} = \lambda_{t+1}\mathbf{1}_{|\mathcal{N}_t|}, \qquad \mathbf{n}_{t+1} = [n^{(1)}, \ldots, n^{(|\mathcal{N}_t|)}]^\top.$$

The stacked predictor input is $\bar{\mathbf{S}}_t = [\,\mathbf{S}_t, \boldsymbol{\lambda}_{t+1}, \mathbf{n}_{t+1}\,] \in \mathbb{R}^{|\mathcal{N}_t|\times(d+2)}$. A single tensorized forward pass then yields $\mathbf{S}^o_{t+1} = \mathcal{F}(\bar{\mathbf{S}}_t) \in \mathbb{R}^{|\mathcal{N}_t|\times d}$, which contains the predicted next-period states for all candidate staffing levels simultaneously.

Staffing Selection Rule. Let $(\mathbf{S}^o_{t+1})_i$ denote the ith row of $\mathbf{S}^o_{t+1}$. The feasible staffing set at decision epoch t is defined as $\Phi_t = \left\{ n^{(i)} \in \mathcal{N}_t \;\middle|\; \Pi\left((\mathbf{S}^o_{t+1})_i\right) \le 0 \right\}$. The adaptive staffing decision is then

$$n^{\mathrm{AS}}_{t+1} = \begin{cases} \min \Phi_t, & \text{if } \Phi_t \neq \varnothing, \\ \bar{n}, & \text{otherwise.} \end{cases}$$

Hence, when an adjustment is required, QUANT-AS selects the minimum candidate staffing level predicted to satisfy all prescribed constraints at the current decision epoch. This is a period-by-period feasibility rule; it does not imply global optimality of the resulting staffing trajectory.

5.2. Experimental Configuration

We construct 200 distinct operational scenarios drawn from the experimental configuration of Section 3, including the three different distributional configurations presented there. We evaluate the performance of QUANT-AS under three classes of time dependent service level constraints: mean waiting time $\mathbb{E}[W(t)] \le \bar{w}$, mean queue length $\mathbb{E}[Q(t)] \le \bar{q}$, and TPoD constraints $\mathbb{P}(W(t) > w) \le \alpha$. Complete parameter specifications are in Appendix F.

5.3. Computational Performance

QUANT-AS successfully identifies a feasible staffing trajectory in all tested instances and completes each staffing run in approximately 0.8 seconds using batch evaluation, resulting in negligible computational overhead. We also consider binary-search and gradient-based alternative staffing procedures, with methodological details and comparative results reported in Appendix G. All methods benefit from QUANT's fast inference, requiring approximately 1 millisecond per forward pass at each time step.

5.4. Validation Against Simulation

We next validate whether the staffing trajectories generated by QUANT-AS satisfy the prescribed SL constraints when evaluated in the underlying queueing system. For each scenario, QUANT-AS uses QUANT predictions to make the sequential staffing decisions described in Section 5.1. We then simulate the corresponding $G_t/GI/n_t + GI$ system under the resulting staffing trajectory and evaluate constraint satisfaction using the simulated system performance.

Algorithm 1 QUANT-AS: Adaptive Staffing via Batch Evaluation

Require: Horizon T; initial system state $\mathbf{s}_0 \in \mathbb{R}^d$; arrival-rate trajectory $\{\lambda_t\}_{t=1}^T$; baseline staffing $\{n_t\}_{t=1}^T$; QUANT model $\mathcal{F}(\cdot)$; constraint policy $\Pi(\cdot)$; staffing bounds $(\underline{n}, \bar{n})$

Ensure: Adaptive staffing trajectory $\{n_t^{\mathrm{AS}}\}_{t=1}^T$

for $t = 0, \ldots, T-1$ **do**
  $\bar{\mathbf{S}}_t^b \leftarrow \left[\mathbf{s}_t^\top \ \lambda_{t+1} \ n_{t+1}\right]$
  $\mathbf{S}_{t+1}^b \leftarrow \mathcal{F}(\bar{\mathbf{S}}_t^b)$ ▷ Evaluate baseline staffing
  if $\Pi(\mathbf{S}_{t+1}^b) \le 0$ **then**
    $n_{t+1}^{\mathrm{AS}} \leftarrow n_{t+1}$ ▷ Retain feasible baseline staffing
  else
    $\mathcal{N}_t \leftarrow \{\underline{n}, \underline{n}+1, \ldots, \bar{n}\}$, $\mathbf{S}_t \leftarrow \mathbf{1}_{|\mathcal{N}_t|}\mathbf{s}_t^\top$, $\boldsymbol{\lambda}_{t+1} \leftarrow \lambda_{t+1}\mathbf{1}_{|\mathcal{N}_t|}$, $\mathbf{n}_{t+1} \leftarrow [n^{(1)}, \ldots, n^{(|\mathcal{N}_t|)}]^\top$
    $\bar{\mathbf{S}}_t \leftarrow [\,\mathbf{S}_t, \boldsymbol{\lambda}_{t+1}, \mathbf{n}_{t+1}\,]$
    $\mathbf{S}_{t+1}^o \leftarrow \mathcal{F}(\bar{\mathbf{S}}_t)$ ▷ Evaluate all candidates in parallel
    $\Phi_t \leftarrow \left\{(\mathbf{n}_{t+1})_i \mid \Pi((\mathbf{S}_{t+1}^o)_i) \le 0\right\}$
    if $\Phi_t \neq \emptyset$ **then**
      $n_{t+1}^{\mathrm{AS}} \leftarrow \min \Phi_t$
    else
      $n_{t+1}^{\mathrm{AS}} \leftarrow \bar{n}$
    end if
  end if
  $\bar{\mathbf{S}}_t^{\mathrm{AS}} \leftarrow \left[\mathbf{s}_t^\top \ \lambda_{t+1} \ n_{t+1}^{\mathrm{AS}}\right]$
  $\mathbf{S}_{t+1}^{\mathrm{AS}} \leftarrow \mathcal{F}(\bar{\mathbf{S}}_t^{\mathrm{AS}})$ ▷ Update state using selected staffing
  $\mathbf{s}_{t+1} \leftarrow (\mathbf{S}_{t+1}^{\mathrm{AS}})^\top$
end for
return $\{n_t^{\mathrm{AS}}\}_{t=1}^T$

Table 3 compares QUANT's prediction accuracy with the resulting constraint satisfaction in simulation. Simulation success rate is defined as the fraction of time periods in which simulated performance satisfies the specified constraints. For each scenario, baseline simulation under a staffing trajectory $\{n_t\}_{t=1}^T$ is used to calibrate the constraint thresholds. For the mean-based constraints, the thresholds are set to 25%, 50%, and 75% of the corresponding baseline maximum, i.e., $\max_t \mathbb{E}[W(t)]$ or $\max_t \mathbb{E}[Q(t)]$. For the TPoD constraints, we use $w \in \{0.5, 1.0\}$ and $\alpha \in \{0.25, 0.50, 0.75\}$. In the multi-constraint experiments, a mean queue-length constraint and a TPoD constraint are imposed simultaneously, and both constraints must be satisfied at a given time t for that period to be considered feasible.

Table 3 Prediction accuracy and constraint satisfaction in simulation.

Constraint Type	Prediction Error (TAMSE)	Simulation Success Rate
$\mathbb{E}[W(t)] \le \bar{w}$	0.095	97.0%
$\mathbb{E}[Q(t)] \le \bar{q}$	0.088	97.5%
$\mathbb{P}(W(t) > w) \le \alpha$	0.131	95.0%
Multi-constraint	0.140	94.0%

Note: Simulation success rate represents the percentage of time steps at which simulated performance satisfies the prescribed threshold(s).

Although the staffing decisions are based on QUANT-predicted performance, simulation achieves constraint-satisfaction rates between 94.0% and 97.5% across the tested constraint classes. The highest satisfaction rates are observed for the mean waiting-time and mean queue-length constraints, whereas the TPoD and multi-constraint settings are more challenging. In particular, tail-probability metrics are more sensitive to small staffing changes and prediction errors, which can translate into occasional constraint violations when the selected staffing level lies close to the feasibility boundary.

Overall, these results show that the staffing trajectories generated by QUANT-AS maintain high SL-constraint satisfaction when evaluated using simulation of the underlying queueing system. This provides an out-of-model validation of the adaptive staffing procedure: QUANT is used to make the staffing decisions, while simulation is used independently to assess the resulting service performance.

5.5. Robustness Across System Configurations

Table 4 Robustness analysis across system configurations.

Category	Configuration	Simulation Success Rate	Prediction Error (TAMSE)
System Scale	Small	96.5%	0.156
	Medium	97.0%	0.140
	Large	98.5%	0.132
Distribution	$M_t/M/n_t + M$	98.5%	0.121
	$M_t/E_4/n_t + E_4$	97.0%	0.224
	$M_t/H_2/n_t + H_2$	95.5%	0.360

Scale comparisons are conducted under the Markovian setting, and distributional comparisons at the baseline system scale.

Table 4 examines QUANT-AS across different distributional families and system scales, following the same scale classification defined in Section 4.2. Simulation constraint-satisfaction rates remain above 95% for all configurations tested. Across system scales, the satisfaction rate increases from 96.5% for small systems to 98.5% for large systems, accompanied by a reduction in prediction error. This pattern is consistent with the greater relative simulation noise observed at smaller scales.

Across distributional settings, the Markovian system exhibits the highest simulation satisfaction rate and lowest prediction error, while performance degrades moderately for the Erlang and high-variability hyperexponential systems. Nevertheless, QUANT-AS maintains simulation constraint-satisfaction rates of at least 95.5% across all three distributional families. These results indicate that

the adaptive staffing procedure remains effective across the range of system scales and distributional settings considered.

5.6. A Multi-Constraint Adaptive Staffing Example

Practitioners across domains encounter service-level agreements (SLAs) involving multiple service-level constraints. For example, cloud data centers manage request queues under tail-latency guarantees (Dean and Barroso 2013), hospitals make staffing decisions subject to waiting-time and capacity constraints (Bittencourt et al. 2018), and contact centers operate under joint constraints on TPoD and average waiting time (Koole and Mandelbaum 2002). QUANT-AS naturally accommodates such settings by evaluating multiple SL constraints simultaneously through the constraint policy function $\Pi(\cdot)$.

Figure 7 **QUANT-AS under multiple SL targets: performance under the baseline staffing plan (solid line) and the adaptive staffing plan generated by QUANT-AS (dashed line). The dotted lines indicate the queue-length and TPoD service-level limits.**

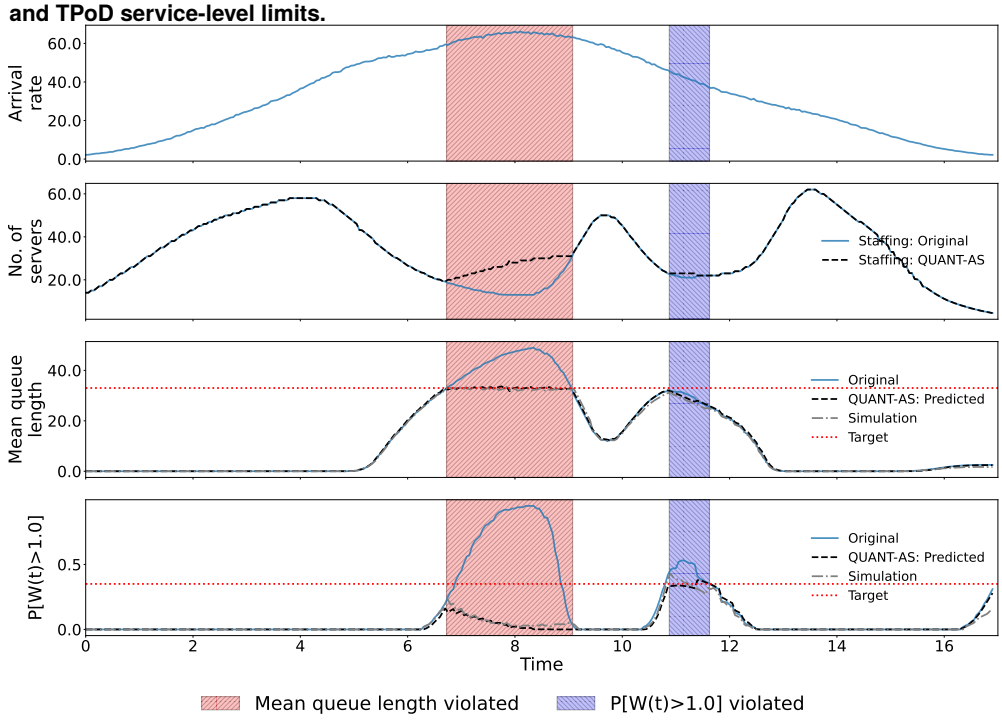


Figure 7 illustrates QUANT-AS under two simultaneous SL constraints: $\mathbb{E}[Q(t)] \le \bar{q} = 33$ and $\mathbb{P}(W(t) > 1) \le \alpha = 0.35$ for all $t \in [0, T]$. The scenario features an arrival pattern that peaks at 70 customers per time unit at $t = 8$ and a baseline staffing trajectory that does not satisfy the prescribed SL constraints over portions of the horizon. Service and abandonment times follow hyperexponential H_2 distributions with mean 1 and SCV = 4, representing a high-variability setting.

When the baseline staffing is predicted to violate one or both constraints, QUANT-AS evaluates alternative staffing levels and adjusts capacity according to the period-by-period selection rule in Section 5.1. The resulting adaptive staffing trajectory responds to different sources of service-level pressure over the horizon while accounting for both constraints simultaneously. Simulation of the resulting queue confirms a 96.8% constraint-satisfaction rate across the adjusted periods (61 of 63), despite the high distributional variability and simultaneous SL requirements.

This example illustrates an important practical feature of QUANT-AS: different service-level metrics may become binding at different points in time, yet they can be incorporated within the same adaptive staffing procedure through a common constraint policy.

6. Conclusions

This paper introduced QUANT, a transfer-learning framework for transient performance prediction and adaptive staffing in nonstationary non-Markovian queues. QUANT pretrains a neural network on efficiently generated Markovian data and adapts its predictions using a small simulation dataset from the target non-Markovian system. This approach avoids reliance on pointwise steady-state approximations and supports time-varying staffing and systems without customer abandonment.

Numerical experiments evaluate QUANT across different system scales and service- and abandonment-time distributions. Once trained, QUANT evaluates transient performance orders of magnitude faster than simulation. In the target-data experiment, using 100 rather than 10,000 simulated trajectories yields similar prediction accuracy, illustrating the data efficiency of the transfer-learning approach. QUANT-based Adaptive Staffing (QUANT-AS) uses these fast predictions to adjust staffing sequentially under multiple service-level constraints. Simulation validation yields constraint-satisfaction rates of 94.0%–98.5% across the tested settings.

Limitations and Future Directions. First, adaptation currently requires simulation data for each specified pair of service- and abandonment-time distributions. Extending transfer across distributional families could reduce this requirement. Second, QUANT-AS makes sequential staffing adjustments rather than jointly optimizing costs and service performance over the planning horizon. Future work could incorporate the effects of current decisions on future congestion, staffing adjustment costs, operational constraints, and feedback from real-time observations.

References

Aras AK, Chen X, Liu Y (2018) Many-server gaussian limits for non-markovian queues with customer abandonment. *Queueing Systems* 89(1):81–125.

Baron O, Krass D, Senderovich A, Sherzer E (2024) Supervised ML for solving the GI/GI/1 queue. *INFORMS Journal on Computing* 36(3):766–786, URL http://dx.doi.org/10.1287/ijoc.2022.0263.

Baron O, Krass D, Sherzer E, Senderovich A (2022) Can machines solve general queueing problems? *Proceedings of the 2022 Winter Simulation Conference (WSC)*, 2830–2841 (IEEE), URL http://dx.doi.org/10.1109/WSC57314.2022.10015451.

Bilancia M, Manca F, Sansaro G (2021) *A Time Series Decomposition Algorithm Based on Gaussian Processes*, 577–592 (Springer International Publishing).

Bittencourt O, Verter V, Yalovsky M (2018) Hospital capacity management based on the queueing theory. *International Journal of Productivity and Performance Management* 67(2):224–238.

Brown L, Gans N, Mandelbaum A, Sakov A, Shen H, Zeltyn S, Zhao L (2005) Statistical analysis of a telephone call center: A queueing-science perspective. *Journal of the American Statistical Association* 100(469):36–50.

Caruana R (1997) Multitask learning. *Machine learning* 28(1):41–75.

Dean J, Barroso LA (2013) The tail at scale. *Commun. ACM* 56(2):74–80.

Defraeye M, Van Nieuwenhuyse I (2013) Controlling excessive waiting times in small service systems with time-varying demand: An extension of the isa algorithm. *Decision Support Systems* 54(4):1558–1567.

Feldman Z, Mandelbaum A, Massey WA, Whitt W (2008) Staffing of time-varying queues to achieve time-stable performance. *Management Science* 54(2):324–338.

Garyfallos S, Liu Y, Barlet-Ros P, Cabellos-Aparicio A (2025) NeuraliNQ: a neural network method for the transient performance analysis in non-markovian queues. *Queueing Systems* 109(4), URL http://dx.doi.org/10.1007/s11134-025-09952-3.

Green L, Kolesar P, Whitt W (2007) Coping with time-varying demand when setting staffing requirements for a service system. *Production Oper. Management* 16(1):13–39.

He B, Liu Y, Whitt W (2016) Staffing a service system with non-Poisson nonstationary arrivals. *Probab. Engrg. Inform. Sci.* 30(4):593–621.

Jennings OB, Mandelbaum A, Massey WA, Whitt W (1996) Server staffing to meet time-varying demand. *Management Science* 42(10):1383–1394.

Konrad K, Liu Y (2023) Achieving stable service-level targets in time-varying service systems: A simulation-based offline learning staffing algorithm. *2025 Winter Simulation Conference (WSC)*, 327–338.

Koole G, Mandelbaum A (2002) Queueing models of call centers: An introduction. *Annals of Operations Research* 113(1–4):41–59.

Liu Y (2018) Staffing to stabilize the tail probability of delay in service systems with time-varying demand. *Operations Research* 66(2):514–534.

Liu Y, Sun X, Hovey K (2022) Scheduling to differentiate service in a multiclass service system. *Operations Research* 70(1):527–544.

Liu Y, Whitt W (2011) A network of time-varying many-server fluid queues with customer abandonment. *Operations Research* 59(4):835–846.

Liu Y, Whitt W (2012a) The $G_t/GI/s_t + GI$ many-server fluid queue. *Queueing Systems* 71(4):405–444.

Liu Y, Whitt W (2012b) Stabilizing customer abandonment in many-server queues with time-varying arrivals. *Operations Research* 60:1551–1564.

Liu Y, Whitt W (2014a) A many-server heavy-traffic limit for queues with time-varying parameters. *The Annals of Applied Probability* 24(1):61–126.

Liu Y, Whitt W (2014b) Stabilizing performance in network of queues with time-varying arrival rates. *Probab. Engrg. Inform. Sci.* 28(4):419–449.

Liu Y, Whitt W (2017) Stabilizing performance in many-server queues with time-varying arrivals and customer feedback. *Eur. J. Oper. Res.* 256(2):473–486.

Liu Y, Whitt W, Yu Y (2016) Approximations for heavily-loaded $G/GI/n + GI$ queues. *Naval Research Logistics* 63:187–217.

Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for Markovian service networks. *Queueing systems* 30(1-2):149–201.

Meng X, Karniadakis GE (2020) A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse pde problems. *Journal of Computational Physics* 401:109020.

Pan SJ, Yang Q (2010) A survey on transfer learning. *IEEE Transactions on knowledge and data engineering* 22(10):1345–1359.

Perdikaris P, Raissi M, Damianou A, Lawrence ND, Karniadakis GE (2017) Nonlinear information fusion algorithms for data-efficient multi-fidelity modelling. *Proceedings. Mathematical, physical, and engineering sciences* 473(2198):20160751.

Pratt LY, Mostow J, Kamm CA (1991) Direct transfer of learned information among neural networks. *Proceedings of the ninth national conference on artificial intelligence* 584–589.

Sherzer E, Baron O, Krass D, Resheff Y (2025) Approximating G(t)/GI/1 queues with deep learning. *European Journal of Operational Research* 322(3):889–907.

Shi P, Chou M, Dai JG, Ding D, Sim J (2014) Models and insights for hospital inpatient operations: Time-dependent ed boarding time. *Management Sci.* 62(1):1–28.

Sun X, Liu Y (2021) Staffing many-server queues with autoregressive inputs. *Naval Research Logistics* 68(3):312–326.

Whitt W (2006) A multi-class fluid model for a contact center with skill-based routing. *Aeu-international Journal of Electronics and Communications* 60:95–102.

Whitt W (2018) Time-varying queues. *Queueing models and service management* 1(2):79–164.

APPENDIX

Appendix A: Nomenclature

Table 5 summarizes the acronyms and terminology used throughout the paper.

Category	Acronym	Description
Problem setting	NSNM	Nonstationary non-Markovian
Proposed in this work	QUANT	QUeue Analysis via Neural-network Transfer learning
	MQNet	Markovian Queue Network
	NMQT	non-Markovian Queue Transformer
	QUANT-AS	QUANT-based Adaptive Staffing
	MQSolver	Markovian Queue Solver
Metrics	SL	Service Level
	SLAs	Service Level Agreements
	TAMSE	Time-Averaged Mean Squared Error
	TPoD	Tail Probability of Delay
	PoD	Probability of Delay
	PoA	Probability of Abandonment
	PWT	Potential Waiting Time
	AWT	Actual Waiting Time
Prior methods	NeuraliNQ	Neural-network-based prediction method for non-Markovian queue
	PSS	Pointwise Steady State
	MOL	Modified-offered-load
	ISA	Simulation-Based Iterative Staffing Algorithm
Queueing primitives	FCFS	First-come, First-Served
Heavy-traffic limits	FCLT	Functional Central Limit Theorem
	FLLN	Funcitonal Law of Large Numbers
General ML components	RNN	Recurrent Neural Network

Table 5 Summary of acronyms and terminology

Appendix B: MQSolver: Complete Algorithmic Details

This appendix provides the complete algorithmic details for MQSolver, the Markovian queue performance oracle introduced in Section 3.2.1. MQSolver computes exact transient performance metrics for the $M_t/M/n_t+M$ queueing system by exploiting its Markovian structure through uniformization techniques.

B.1. Queue-Length Distribution

We discretize the total time horizon T into M small successive intervals of length $h = T/M$. For a sufficiently small h, at most one event can occur in each interval, which corresponds to one of the following events: arrival of a new customer or departure of an existing customer (either due to service completion or abandonment).

To ensure computational feasibility, we truncate the state space at a maximum queue length N (a large number). At each discrete time step $t_k = k \cdot h$ for $k = 0, 1, \ldots, M-1$, the total queue length evolves according to a birth-and-death process with a finite state space $\mathcal{S} = \{0, 1, \ldots, N\}$. When $X(t_k) = i$ for $i \in \mathcal{S}$, the system makes transitions with the following probabilities:

$$P_{i,j}(t_k) \equiv \mathbb{P}(X(t_{k+1}) = j | X(t_k) = i)$$
$$= \begin{cases} \lambda(t_k)h, & j = i+1 \quad \text{(An arrival)} \\ (n_{t_k} \wedge i)\mu h + (i - n_{t_k})^+ \theta h, & j = i-1 \quad \text{(A departure)} \\ 1 - \lambda(t_k)h - (n_{t_k} \wedge i)\mu h - (i - n_{t_k})^+ \theta h, & j = i \quad \text{(No event)} \\ 0, & \text{otherwise,} \end{cases}$$

where $a \wedge b \equiv \min(a, b)$ and $a^+ \equiv \max(a, 0)$.

We denote $\boldsymbol{\pi}(t_k) \equiv [\pi_0(t_k), \pi_1(t_k), \ldots, \pi_N(t_k)]$ as the transient distribution of $X(t_k)$, that is, $\pi_i(t_k) \equiv \mathbb{P}(X(t_k) = i)$. Starting in a given initial distribution $\boldsymbol{\pi}(0)$ (e.g., $\boldsymbol{\pi}(0) = [1, 0, \ldots, 0]$ indicating an empty system), the transient distribution at all time steps can be obtained via matrix multiplications:

$$\boldsymbol{\pi}(t_k) = \boldsymbol{\pi}(0)\mathbf{P}(t_0)\mathbf{P}(t_1)\cdots\mathbf{P}(t_{k-1}), \tag{13}$$

where $\mathbf{P}(t_k)$ is the one-step transition probability matrix with its $(i, j)^{\text{th}}$ element being $P_{i,j}(t_k)$.

Once we have the queue-length distribution, we can immediately compute the queue-based service-level metrics such as the mean total queue length $\mathbb{E}[X(t_k)]$, mean length of the waiting queue $\mathbb{E}[Q(t_k)]$, the server utilization $\mathbb{E}[B(t_k)]/n_{t_k}$ (the ratio of mean number of busy servers $B(t_k)$ to the total number of servers), and rates of abandonment $\alpha(t_k)$ and service completion $\sigma(t_k)$, at all time steps t_k, for $k = 0, \ldots, M-1$:

$$\begin{aligned} &\mathbb{E}[X(t_k)] = \sum_{i=0}^{N} i \cdot \pi_i(t_k), \quad \mathbb{E}[Q(t_k)] = \sum_{i=0}^{N} (i - n_{t_k})^+ \cdot \pi_i(t_k), \\ &\frac{\mathbb{E}[B(t_k)]}{n_{t_k}} = \frac{1}{n_{t_k}} \sum_{i=0}^{N} \left(i \wedge n_{t_k}\right) \cdot \pi_i(t_k), \quad \alpha(t_k) = \theta \mathbb{E}[Q(t_k)], \quad \sigma(t_k) = \mu \mathbb{E}[B(t_k)]. \end{aligned} \tag{14}$$

B.2. Waiting Times Conditional on the Queue Length

To compute the waiting time distribution, we first condition on the total number of customers in the system at arrival time. We first treat the potential waiting time (PWT) $W(t)$, the offered waiting time for a customer arrival at t (if any) assuming the customer is infinitely patient.

If $x < n_{t_k}$, we have $W(t_k) = 0$. If $x \geq n_{t_k}$, then all servers are busy and there are $q = x - n_{t_k} \geq 0$ customers waiting in the queue. Let $W_q(t_k)$ be the waiting time of a customer arrival (which we call the "tagged customer") conditional on q waiting customers at t_k.

We model $W_q(t_k)$ by a phase-type distribution, which is the time for a pure death process to reach an absorbing state 0, beginning at an initial state $q+1$. Specifically, the state space is $\mathcal{S}_{t_k,q} \equiv \{0, 1, \ldots, q+1\}$ where $j \in \mathcal{S}_{t_k,q}$ tracks the customer's queueing position after time t_k, with $q+1$ being the initial position at t_k and 0 meaning the customer first entering service at the future time $t_k + W_q(t_k)$. Here, a "death" corresponds to a departure from the waiting line due to abandonment or a service completion from n_{t_k} busy servers.

We again consider a discrete-time framework. After a sufficiently small time step h, the tagged customer's queue position decreases by 1 (i.e., moving towards the head of the line) with probability $(n_{t_k}\mu + (j-1)\theta)h$ when at position j, while its queueing position remains unchanged with probability $1 - (n_{t_k}\mu + (j-1)\theta)h$. Hence, for row j where

$1 \le j \le q+1$, the transition probability to column $j-1$ is $(n_{t_k}\mu + (j-1)\theta)h$ and to column j is $1-(n_{t_k}\mu+(j-1)\theta)h$, with all other entries being zero. The transition probability matrix is:

$$\begin{array}{c|c||ccccc|c} & 0 & 1 & 2 & \cdots & q & q+1 \\ \hline\hline 0 & 1 & 0 & 0 & \cdots & 0 & 0 \\ \hline 1 & n_{t_k}\mu h & 1-n_{t_k}\mu h & 0 & \cdots & 0 & 0 \\ 2 & 0 & (n_{t_k}\mu+\theta)h & 1-(n_{t_k}\mu+\theta)h & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ q & 0 & 0 & 0 & \cdots & 1-(n_{t_k}\mu+(q-1)\theta)h & 0 \\ q+1 & 0 & 0 & 0 & \cdots & (n_{t_k}\mu+q\theta)h & 1-(n_{t_k}\mu+q\theta)h \end{array} \tag{15}$$

Let $\mathbf{P}_{t_k}(q)$ denote the $(q+1)\times(q+1)$ submatrix excluding the 0th row and column. When the staffing level changes between time steps, we must also account for instantaneous repositioning. If staffing increases from $n_{t'_{i-1}}$ to $n_{t'_i}$ by $\Delta n = n_{t'_i} - n_{t'_{i-1}} > 0$, then Δn customers at the head of the queue immediately enter service, causing each remaining customer's position to decrease by Δn. We capture this through a repositioning matrix $\mathbf{R}_i(\Delta n)$ where $R_{j,k} = 1$ if $k = (j-\Delta n)^+$ and $R_{j,k} = 0$ otherwise. For staffing unchanged or decreased, $\mathbf{R}_i = \mathbf{I}$ (the identity matrix).

For $\tau > 0$, $q \ge 0$, and $t'_i \equiv t_k + ih$, $M_\tau \equiv \lceil \tau/h \rceil$, the conditional waiting time distribution given $X(t_k) = x$ is:

$$\mathbb{P}(W(t_k) \le \tau \mid X(t_k) = x) = \begin{cases} 1, & x < n_{t_k}, \\ 1 - \boldsymbol{\beta}_x \prod_{i=1}^{M_\tau} \left[\mathbf{R}_i \cdot \mathbf{P}_{t'_i}(x - n_{t_k})\right] \mathbf{e}, & x \ge n_{t_k}, \end{cases} \tag{16}$$

where $\boldsymbol{\beta}_x \equiv [0, \cdots, 0, 1]$ is a row vector with q zeros and a 1 in its last cell, and $\mathbf{e} \equiv [1, \cdots, 1]^T$ is a column vector of all ones.

Using the queue-length distribution derived in Section B.1, we retrieve the unconditional waiting-time distribution:

$$\mathbb{P}(W(t_k) \le \tau) = \sum_{x=0}^{N} \mathbb{P}(W(t_k) \le \tau \mid X(t_k) = x)\pi_x(t_k), \tag{17}$$

from which the tail probability of delay $\mathbb{P}(W(t_k) > \tau) = 1 - \mathbb{P}(W(t_k) \le \tau)$ follows immediately.

B.3. Mean Waiting Time via Backward Recursion

The mean waiting time $\mathbb{E}[W(t_k)]$ can be obtained using the waiting-time distribution in (17) at different τ. We hereby give a more efficient way to compute $\mathbb{E}[W(t_k)]$ using backward recursion.

For $t_k = kh$, we define $\mathbb{E}[W(t_k, q)]$ as the expected remaining waiting time at time t_k for a customer who has a queueing position $q > 0$ in the waiting line. Let $\Delta n_k = (n_{t_{k+1}} - n_{t_k})^+$ denote the staffing increase (zero if unchanged or decreased). The recursion depends on whether the customer enters service due to the staffing change.

If $q \le \Delta n_k$, the customer immediately enters service and $\mathbb{E}[W(t_k, q)] = h$. Otherwise, let $q' = q - \Delta n_k$ denote the new position (with $q' = q$ when $\Delta n_k = 0$). The recursion is:

$$\mathbb{E}[W(t_k, q)] = h + (n_{t_{k+1}}\mu + (q'-1)\theta)h \cdot \mathbb{E}[W(t_{k+1}, q'-1)] + (1 - (n_{t_{k+1}}\mu + (q'-1)\theta)h) \cdot \mathbb{E}[W(t_{k+1}, q')], \tag{18}$$

where the first term accounts for advancing one position (service completion or abandonment of a customer ahead) and the second term accounts for remaining in position. The recursion implicitly assumes $\mathbb{E}[W(t_k, 0)] = 0$ for all t_k.

At $q = 1$ with no staffing increase ($\Delta n_k = 0$), the boundary condition is:

$$\mathbb{E}[W(t_k, 1)] = h + (1 - n_{t_{k+1}}\mu h) \cdot \mathbb{E}[W(t_{k+1}, 1)], \tag{19}$$

since the tagged customer only needs to wait for a service completion. If $\Delta n_k \geq 1$, then $\mathbb{E}[W(t_k, 1)] = h$.

At the end of the time horizon, assuming the system reaches a stationary regime with constant staffing n_T (i.e., $\mathbb{E}[W(T, q)] = \mathbb{E}[W(T + h, q)]$), we have:

$$\mathbb{E}[W(T, q+1)] = \frac{1}{n_T\mu + q\theta} + \mathbb{E}[W(T, q)], \tag{20}$$

with base case $\mathbb{E}[W(T, 1)] = \frac{1}{n_T\mu}$. The unconditional mean waiting time is:

$$\mathbb{E}[W(t_k)] = \sum_{x=n_{t_k}}^{N} \mathbb{E}[W(t_k, x - n_{t_k} + 1)] \cdot \pi_x(t_k). \tag{21}$$

B.4. Actual Waiting Time (AWT)

The above analysis can be extended to the actual waiting time (AWT) $W_a(t) \equiv \min(W(t), A)$, with A denoting a generic abandonment time. Unlike PWT, AWT accounts for the possibility that the tagged customer abandons before reaching service.

For the distribution of AWT, we modify the transition probability matrix (15) to include the tagged customer's own abandonment. For row j where $1 \leq j \leq q+1$, the non-zero entries are: column 0 has θh (tagged customer abandons), column $j-1$ has $(n_{t_k}\mu + (j-1)\theta)h$ (service completion or abandonment of a customer ahead), and column j has $1 - (n_{t_k}\mu + j\theta)h$ (no event). For row 1, the entries for columns 0 and 1 are $(n_{t_k}\mu + \theta)h$ and $1 - (n_{t_k}\mu + \theta)h$ respectively.

	0	1	2	$\cdots$	$q-1$	q	$q+1$
0	1	0	0	$\cdots$	0	0	0
1	$(n_{t_k}\mu+\theta)h$	$1-(n_{t_k}\mu+\theta)h$	0	$\cdots$	0	0	0
2	θh	$(n_{t_k}\mu+\theta)h$	$1-(n_{t_k}\mu+2\theta)h$	$\cdots$	0	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$
q	θh	0	0	$\cdots$	$(n_{t_k}\mu+(q-1)\theta)h$	$1-(n_{t_k}\mu+q\theta)h$	0
$q+1$	θh	0	0	$\cdots$	0	$(n_{t_k}\mu+q\theta)h$	$1-(n_{t_k}\mu+(q+1)\theta)h$

(22)

The conditional distribution for AWT can be computed using (16) with the submatrix from (22).

For the mean AWT, the recursion follows the same structure as PWT. If $q \leq \Delta n_k$, then $\mathbb{E}[W_a(t_k, q)] = h$. Otherwise, with $q' = q - \Delta n_k$:

$$\mathbb{E}[W_a(t_k, q)] = h + (n_{t_{k+1}}\mu + (q'-1)\theta)h \cdot \mathbb{E}[W_a(t_{k+1}, q'-1)] + (1 - (n_{t_{k+1}}\mu + q'\theta)h) \cdot \mathbb{E}[W_a(t_{k+1}, q')], \tag{23}$$

where the "stay" probability includes the tagged customer's own abandonment rate.

At $q = 1$ with no staffing increase, $\mathbb{E}[W_a(t_k, 1)] = h + (1 - (n_{t_{k+1}}\mu + \theta)h) \cdot \mathbb{E}[W_a(t_{k+1}, 1)]$. If $\Delta n_k \geq 1$, then $\mathbb{E}[W_a(t_k, 1)] = h$.

At the end of the time horizon under stationarity:

$$\mathbb{E}[W_a(T, q+1)] = \frac{1}{n_T\mu + (q+1)\theta} + \frac{n_T\mu + q\theta}{n_T\mu + (q+1)\theta} \cdot \mathbb{E}[W_a(T, q)], \tag{24}$$

with base case $\mathbb{E}[W_a(T, 1)] = \frac{1}{n_T\mu + \theta}$.

Appendix C: Data Generation Procedures

This appendix provides complete details of the data generation procedures described in Section 3.2.3.

C.1. Input Sampling Strategy

To generate our training dataset, we use the temporal profile $b(t)$ defined in Section 3.2.2. Throughout the experiments presented in this paper, we set $K = 10$ Gaussian components. For each component $k = 1, \ldots, K$, the parameters c_k, μ_k, and σ_k are independently drawn from uniform distributions. For the arrival-rate trajectory $\lambda(t)$, we sample $\mu_k \sim \mathcal{U}(3, 14)$, $\sigma_k \sim \mathcal{U}(0.4, 3.0)$, and $c_k \sim \mathcal{U}(10, 100)$. For the service-capacity trajectory $n(t)$, we sample $\mu_k \sim \mathcal{U}(1, 16)$, $\sigma_k \sim \mathcal{U}(0.4, 3.0)$, and $c_k \sim \mathcal{U}(10, 100)$. For each trajectory, the maximum level M is independently sampled from $\mathcal{U}(10, 100)$, after which the profile is rescaled as $b(t) = \frac{M}{\max_{s\in[0,T]} P(s)} P(t)$. Finally, we bias the training data toward overloaded combinations of $\lambda(t)$ and $n(t)$ by retaining trajectories satisfying $\frac{\sum_{j=0}^{J}[\lambda(t_j)-n(t_j)]}{\sum_{j=0}^{J}\lambda(t_j)} > 0.3$, where $t_j = j\Delta t$, $j = 0, \ldots, J$, and Δt denotes the time step. This filtering criterion removes trajectories that are predominantly overstaffed and would otherwise yield mostly zero or near-zero queueing-performance targets, providing limited information for model training.

C.2. Simulation Procedures

For each simulation sample, we begin with an initially empty system and obtain the Monte Carlo estimates of the performance trajectories by averaging over N_s independent simulation paths. Throughout the experiments, we set $N_s = 10{,}000$, targeting a standard error of approximately 0.01 or less for each learned performance metric. For both simulation and exact computations, we consider a finite time horizon $[0, T]$ with $T = 17$ and discretize it using a step size of $\Delta t = 0.05$. The resulting time grid is given by $t_j = j\Delta t$, $j = 0, \ldots, J$, where $J = T/\Delta t = 340$.

Appendix D: Training Procedures and Hyperparameters

This appendix provides complete training details for the neural networks described in Section 3.2.4.

To achieve better network convergence, we train our two neural networks separately.

MQNet Training: For $\mathcal{F}^Q$, we use 10,000 sampled input trajectories of $\lambda(t)$ and $n(t)$ generated according to the procedure in Appendix C, with 20% held out for testing. The network is trained for 30 epochs with a batch size of 5,000, a learning rate of 10^{-4}, a hidden width of 5,000 neurons, and a weight decay of 0.01. For $\mathcal{F}^W$, we use 30 epochs, a batch size of 2,000, a learning rate of 10^{-4}, a hidden width of 2,000 neurons, and a weight decay of 0.01. $\mathcal{F}^Q$ and $\mathcal{F}^W$ contain approximately 150 million and 24 million parameters, respectively, and are both trained using the L1 loss and the Adam optimizer.

NMQT Training: For $\mathcal{G}^Q$ and $\mathcal{G}^W$, we use 100 non-Markovian simulation trajectories, corresponding to 1% of the MQNet training data. The networks are fine-tuned for 200 epochs with a batch size of 100, a learning rate of 10^{-7}, and a weight decay of 0.01. We consider three transfer-learning strategies: (a) freezing the first 12 layers of MQNet and fine-tuning the last 4 layers, (b) fine-tuning the entire network with the reduced learning rate, and (c) freezing MQNet and adding NMQT-specific layers with a hidden width of 2,000 neurons. The combined MQNet and NMQT architecture contains approximately 180 million parameters. As in MQNet training, we use the L1 loss and the Adam optimizer.

Appendix E: Prediction Accuracy under Non-Markovian Distributions

This appendix provides additional details on the evaluation of QUANT's prediction accuracy under non-Markovian service and abandonment distributions, complementing the results reported in Section 4.2.

To test QUANT's robustness on NSNM queues with general service and abandonment distributions, we consider hyperexponential H_2 and Erlang (with 4 phases) E_4 with density functions:

$$g_{H_2}(x) = p\mu_1 e^{-\mu_1 x} + (1-p)\mu_2 e^{-\mu_2 x} \quad \text{and} \quad g_{E_4}(x) = \frac{(4\mu)^4 x^3 e^{-4\mu x}}{6},$$

with $p = 0.5(1-\sqrt{0.6})$, $\mu_1 = 1-\sqrt{0.6}$, and $\mu_2 = 1+\sqrt{0.6}$ for H_2, and $\mu = 1$ for E_4. All three cases (M, H_2 and E_4) have equal means. To measure the distribution beyond the mean, we consider the *squared coefficient of variation* (SCV) c^2: we set $c^2 = 4$ and 1/4 for H_2 and E_4 respectively. This is to clearly separate from the base Markovian case where $c^2 = 1$ for exponential distribution.

Appendix F: Adaptive Staffing Experiment Details

This appendix provides complete details of the experimental configuration for the adaptive staffing experiments presented in Section 5.

F.1. Test Scenarios

We construct 200 distinct operational scenarios spanning horizons of $T = 17$ time units with $\Delta t = 0.05$ discretization, matching the temporal configuration from Section 3. The arrival patterns are generated using the basis function approach from Section 3.2.2, yielding diverse temporal profiles including single-peak, double-peak, multi-modal, and high-volatility patterns.

Service and abandonment distributions span three families matching those analyzed in Section 3: $M_t/M/n_t + M$ (Markovian, SCV = 1), $M_t/E_4/n_t + E_4$ (Erlang-4, SCV = 0.25), and $M_t/H_2/n_t + H_2$ (hyperexponential, SCV = 4). All systems maintain mean service time $1/\mu = 1$ and mean abandonment time $1/\theta = 1$. Complete distributional specifications are provided in Appendix E.

F.2. Validation Methodology

To validate QUANT-AS effectiveness, we employ a two-stage verification process as described in Section 5.4. First, we run QUANT-AS given an arrival pattern $\{\lambda_t\}_{t=1}^T$, predetermined staffing $\{n_t\}_{t=1}^T$, and constraint policy $\Pi(\cdot)$ to obtain an adaptive staffing schedule $\{n_t^{AS}\}_{t=1}^T$ and QUANT's predicted performance trajectories.

Second, we simulate the actual $G_t/G/n_t + GI$ system using the staffing schedule generated by QUANT-AS with $N_s = 1,000$ independent replications, following the simulation procedures detailed in Appendix C. The number of replications is lower than the simulated training data of QUANT as we are less interested in random fluctuations of simulation in this validation. We declare a constraint satisfied if the simulated metric remains within the specified bounds for at least 95% of the time steps in the operational horizon $[0, T]$.

For scenarios with multiple constraints, we require all constraints to be simultaneously satisfied at each time step. The reported constraint violation rates in Section 5 represent the percentage of time steps where at least one constraint is violated in the simulated performance.

Appendix G: Alternative Optimization Methods

This appendix presents two alternative optimization methods for the staffing problem and provides a comparative discussion of their empirical performance. Algorithm 1 in Section 5.1 is referred to as the batch evaluation method. Here we describe two alternative optimization approaches that can be implemented by modifying the search procedure (lines 7-15 of Algorithm 1).

G.1. Binary Search Implementation

The binary search method exploits monotonicity (higher staffing yields better performance). We initialize with $n_{low} = \underline{n}$ and $n_{high} = \bar{n}$, then evaluate midpoint $n_{mid} = \lfloor (n_{low} + n_{high})/2 \rfloor$ at each iteration. If $\Pi(\cdot)$ is satisfied at n_{mid}, we update $n^{AS} = n_{mid}$ and $n_{high} = n_{mid}$; otherwise $n_{low} = n_{mid}$. Termination at $n_{high} - n_{low} \leq 1$ guarantees $O(\log_2(\bar{n} - \underline{n}))$ convergence.

For multi-constraint problems, a staffing level is considered feasible only if $\Pi(\mathbf{S}_{t+1}) = \max_i\{\eta_i(t+1) - \bar{\eta}_i\} \leq 0$, ensuring all constraints are simultaneously satisfied.

Algorithm 2 Binary Search Optimization (replaces lines 7-15 of Algorithm 1)

$n_{low} \leftarrow \underline{n}, n_{high} \leftarrow \bar{n}$
while $n_{high} - n_{low} > 1$ **do**
  $n_{mid} \leftarrow \lfloor (n_{low} + n_{high})/2 \rfloor$
  $\bar{\mathbf{S}}_t \leftarrow \left[\mathbf{s}_t^\top \ \lambda_{t+1} \ n_{mid}\right] \in \mathbb{R}^{1\times(d+2)}$
  $\mathbf{S}^o_{t+1} \leftarrow \mathcal{F}\left(\bar{\mathbf{S}}_t\right), \quad \bar{\mathbf{S}}^o_{t+1} \in \mathbb{R}^{1\times d}$
  if $\Pi(\bar{S}^o_{t+1}) \leq 0$ **then**
    $n^{AS}_{t+1} \leftarrow n_{mid}, n_{high} \leftarrow n_{mid}$
  else
    $n_{low} \leftarrow n_{mid}$
  end if
end while

G.2. Gradient-Based Optimization Implementation

We consider the system evolution $S_{t+1} = \mathcal{F}(S_t, \beta_{t+1})$, where the control vector β_{t+1} includes the staffing decision n_{t+1}. Since the transition mapping $\mathcal{F}(\cdot)$ is implemented as a differentiable neural network surrogate, the following computational path is differentiable:

$$n_{t+1} \ \rightarrow \ \beta_{t+1} \ \rightarrow \ S_{t+1} \ \rightarrow \ \Pi(S_{t+1}).$$

As a result, the gradient of the penalty with respect to the staffing decision, $\nabla_{n_{t+1}}\Pi(S_{t+1})$, can be obtained via automatic differentiation (e.g., PyTorch's autograd).

We therefore optimize the surrogate objective $L(n_{t+1}) = \Pi(S_{t+1}) + \epsilon\, n_{t+1}$, using gradient-based updates on a continuous relaxation of n_{t+1}, followed by projection and rounding to obtain an integer staffing decision.

We initialize $\tilde{n} = n_{t+1}$ with gradient tracking enabled, compute the loss and gradient, and update:

$$\tilde{n} \leftarrow \max(\underline{n}, \min(\tilde{n} - \alpha \cdot \nabla_{\tilde{n}} L(\tilde{n}), \bar{n})),$$

where the min and max operations enforce the feasible range $[\underline{n}, \bar{n}]$.

The algorithm terminates when $|\nabla_{\tilde{n}} L(\tilde{n})| < c$ or after I_{max} iterations, then rounds to $n^{AS}_{t+1} = \lfloor \tilde{n} + 0.5 \rfloor$. We use standard gradient descent with fixed learning rate $\alpha = 0.1$, projection onto $[\underline{n}, \bar{n}]$, and early stopping based on gradient magnitude.

Algorithm 3 Gradient-Based Optimization (replaces lines 7-15 of Algorithm 1)

Require: Learning rate α, convergence threshold c, max iterations I_{max}, regularization weight ϵ

$\tilde{n} \leftarrow n_{t+1}$ (enable gradient tracking), $k \leftarrow 0$
while $k < I_{max}$ **do**
  $\bar{\mathbf{S}}_t \leftarrow \left[\mathbf{s}_t^\top \ \lambda_{t+1} \ \tilde{n}\right] \in \mathbb{R}^{1\times(d+2)}$
  $\mathbf{S}^o_{t+1} \leftarrow \mathcal{F}(\bar{\mathbf{S}}_t), \quad \bar{\mathbf{S}}^o_{t+1} \in \mathbb{R}^{1\times d}$
  $L \leftarrow \Pi(\bar{\mathbf{S}}^o_{t+1}) + \epsilon \cdot \tilde{n}$
  if $|\nabla_{\tilde{n}} L(\tilde{n})| < c$ **then break**
  end if
  $\tilde{n} \leftarrow \max(1, \min(\tilde{n} - \alpha \cdot \nabla_{\tilde{n}} L(\tilde{n}), \bar{n}))$
  $k \leftarrow k + 1$
end while
$n^{AS}_{t+1} \leftarrow \lfloor \tilde{n} + 0.5 \rfloor$

G.3. Algorithm Configuration

Table 6 provides the default parameter configuration used across all QUANT-AS experiments. These values are referenced in Algorithm 1 and discussed in Section 5.1.

Table 6 Default QUANT-AS algorithm parameters.

Parameter	Value	Description
$\underline{n}$	1	Minimum feasible staffing level
$\bar{n}$	100	Maximum feasible staffing level
c	0.01	Convergence threshold (gradient-based only)
α	0.1	Learning rate (gradient-based only)
I_{max}	50	Maximum iterations (gradient-based only)
ϵ	0.1	Regularization weight (gradient-based only)

Parameters apply to all three optimization methods unless noted otherwise.

For binary search, bisection continues until the search interval narrows to a single server. For batch evaluation, all candidates within $[\underline{n}, \bar{n}]$ are evaluated in a single vectorized forward pass. For gradient-based optimization, we employ gradient descent with early stopping when the gradient magnitude falls below c or after I_{max} iterations.

Our implementation uses PyTorch for automatic differentiation in the gradient-based method. For binary and batch methods, gradient tracking is disabled to reduce memory overhead. All methods use the same QUANT model checkpoint, ensuring consistent predictions across optimization approaches.

The constraint policy function $\Pi(\mathbf{S}_t)$ is implemented as a modular component, allowing easy extension to new constraint types or combinations.

G.4. Method Comparisons

Table 7 presents convergence characteristics and computational efficiency across optimization methods.

Table 7 QUANT-AS convergence and computational performance.

Method	Iterations per step	Computation time (sec)	Convergence success rate
Binary search	8.2 ± 0.5	1.2	100%
Batch evaluation	1.0 ± 0.0	0.8	100%
Gradient-based	34.5 ± 17.7	10.1	97.2%

Statistics across 200 test scenarios for horizon $T = 17$ (340 time steps).

Batch and binary methods achieve 100% convergence success, completing optimizations in 0.8 and 1.2 seconds respectively. Gradient-based optimization achieves 97.2% success in 10.1 seconds, with occasional failures attributable to suboptimal local minima, subject to hyperparameter tuning.

Appendix H: The $G_t/GI/n_t$ Queueing Systems without Customer Abandonment

This appendix demonstrates QUANT's capability to analyze queueing systems without customer abandonment, as mentioned in Section 1. This represents a key advantage over NeuraliNQ, which required abandonment due to its reliance on pointwise stationary approximation.

Figure 8 shows three queueing systems without customer abandonment, with the same distributional configuration as Figure 2. As expected, such systems experience higher congestion in understaffed situations. Nevertheless, QUANT's neural network structure learns these dynamics effectively, achieving similar accuracy ($\text{TAMSE}_{G_t/G/n_t} = 0.06803$) as systems with abandonment.

Figure 8 QUANT predictions of Markovian and non-Markovian systems without customer abandonment.

